CST1510 Programming for data communication and networks

Week 3: List, Multidimensional Lists, Functions and Lambda Expressions,

WORKSHOP

LISTs (cont.)

Sequence `Slicing`

Slicing is the extraction of a part of a list. It enables access the specific range of elements by mentioning their indices.

Syntax:

- "Start" specifies the starting index of a slice
- "Stop" specifies the ending element of a slice
- You can use one of these if you want to skip certain items

object_name[start : stop : step]

Note: The start index is included but the stop index is ***not included****.*

In [1]:
```
numbers = [2, 3, 5, 7, 11, 13, 17, 19]
# index -  0  1  2  3   4   5   6  7
```

In [2]:
```
numbers[2:6]
```

Out[2]:
```
[5, 7, 11, 13]
```

Activity:

I hope you still have your list of friends... If not, you can use mine:

```
friends = ['Mohamed', 'Olga', 'Roger', 'Iveta','Pawel',
'Aneta','John', 'Joanna' , 'Andrew']
```

- print out the names from 'Olga' to 'Aneta'?

In [ ]:
```
```

Specifying a Slice with Only an Ending Index

- Starting index `0` is assumed.

In [3]:
```
numbers[:6]
```

Out[3]:
```
[2, 3, 5, 7, 11, 13]
```

In [4]:
```
numbers[0:6]
```

Out[4]:
```
[2, 3, 5, 7, 11, 13]
```

Activity:

- From the above list of friend print out the first 3 elements

In [ ]:
```

```

Specifying a Slice with Only a Starting Index

- Assumes the sequence's length as the ending index.

In [5]:
```
numbers[6:]
```

Out[5]:
```
[17, 19]
```

In [6]:
```
numbers[6:len(numbers)]
```

Out[6]:
```
[17, 19]
```

Specifying a Slice with No Indices

In [7]:
```
numbers[:]
```

Out[7]:
```
[2, 3, 5, 7, 11, 13, 17, 19]
```

- While slices create new objects, they produce **shallow copies** of the elements.
- In the example above, the elements of the new list reference the *same objects* as those in the original list.

Activity:

- From the above list of friend print out the last 3 elements

In [ ]:
```

```

Slicing with Steps

In [8]:
```
numbers[::2]
```

Out[8]: [2, 5, 11, 17]

Activity:

From the above list of friends, print out every 2nd element, only the females.

In [ ]:

```

```

Slicing with Negative Indices and Steps

In [9]:

```
numbers = [2, 3, 5, 7, 11, 13, 17, 19]
# index   -8 -7 -6 -5  -4  -3  -2  -1
```

In [10]:

```
numbers[::-1]
```

Out[10]: [19, 17, 13, 11, 7, 5, 3, 2]

In [11]:

```
numbers[-1:-9:-1]
```

Out[11]: [19, 17, 13, 11, 7, 5, 3, 2]

Activity

- Print the first element from your lists
- Print the last element from your lists
- Print the last two elements from your lists

In [ ]:

```

```

Modifying Lists Via Slices

- Can modify a list by assigning to a slice.

In [12]:

```
numbers[0:3] = ['two', 'three', 'five']
```

In [13]:

```
numbers
```

Out[13]: ['two', 'three', 'five', 7, 11, 13, 17, 19]

In [14]:

```
numbers[0:3] = []
```

In [15]:

```
numbers
```

Out[15]: [7, 11, 13, 17, 19]

In [16]:

```
numbers = [2, 3, 5, 7, 11, 13, 17, 19]
```

In [17]:

```
numbers[::2] = [100, 100, 100, 100]
```

In [18]:

```
numbers
```

Out[18]: [100, 3, 100, 7, 100, 13, 100, 19]

In [19]:
```
id(numbers)
```

Out[19]: 4564233536

In [20]:
```
numbers[:] = []
```

In [21]:
```
numbers
```

Out[21]: []

In [22]:
```
id(numbers)
```

Out[22]: 4564233536

- Deleting `numbers`' contents is different from assigning `numbers` a *new* empty list `[]`.
- Identities are different, so they represent separate objects in memory.

In [23]:
```
numbers = []
```

In [24]:
```
numbers
```

Out[24]: []

In [25]:
```
id(numbers)
```

Out[25]: 4564237056

In [ ]:

List Comprehensions

- Concise way to create new lists.
- Replaces using `for` to iterate over a sequence and create a list.

A list comprehension, may require the following ingredients:

- opening and closing square brackets
- function call or an `expression`
- for each loop
- if condition (optional)

Create a List of Integers Using loop

In [26]:
```
list1 = []
```

In [27]:
```
for item in range(1, 6):
    list1.append(item)
```

In [28]:
```
list1
```

Out[28]:
```
[1, 2, 3, 4, 5]
```

Create a List of Integers Using List Comprehension

In [29]:
```
list2 = [item for item in range(1, 6)]
```

In [30]:
```
list2
```

Out[30]:
```
[1, 2, 3, 4, 5]
```

- The `for` **clause** iterates over the sequence generated by `range(1, 6)`.
- For each `item`, the list comprehension evaluates the expression on the left side of the `for` clause and adds its value to the new list.

Activity

Print all names from your list using list comprehension

In [ ]:
```

```

Mapping: Performing Operations in a List Comprehension's Expression

- Mapping is a common functional-style programming operation that produces a result with the *same* number of elements as the original data being mapped.

In [31]:
```
list3 = [item ** 3 for item in range(1, 6)]
```

In [32]:
```
list3
```

Out[32]:
```
[1, 8, 27, 64, 125]
```

List Comprehensions with `if` Clauses

- **Filtering** involves selecting elements that meet a specific condition.
- This usually results in a list containing *fewer* elements than the original data.

In [33]:
```
list4 = [item for item in range(1, 11) if item % 2 == 0]
```

In [34]:
```
list4
```

Out[34]:
```
[2, 4, 6, 8, 10]
```

In [ ]:
```

```

Activity:

- Print out all the names from your list of friends that contain the letter 'a'. Use list comprehension.

In [ ]:
```

```

Two-Dimensional Lists

- Lists can have other lists as their elements.
- They are commonly used to represent **tables** of data, organized in **rows** and **columns**.
- To access a specific element in the table, use *two* indices: the first for the row and the second for the column.

Creating a Two-Dimensional List

In [35]:
```
a = [[77, 68, 86, 73], [96, 87, 89, 81], [70, 90, 86, 81]]
```

Writing the list as follows makes its row and column tabular structure clearer:

```
a = [[77, 68, 86, 73],  # first student's grades
     [96, 87, 89, 81],  # second student's grades
     [70, 90, 86, 81]]  # third student's grades
```

In [36]:
```
# Display all elements from 2d list
print(a)
```

```
[[77, 68, 86, 73], [96, 87, 89, 81], [70, 90, 86, 81]]
```

In [37]:
```
# Display all elements from 1st row
print(a[0])
```

```
[77, 68, 86, 73]
```

In [38]:
```
# Display 1st elements
print(a[0][0])
```

```
77
```

In [43]:
```
# Display all elements from 1st column
for col in a:
    print(col[0])
```

```
77
96
70
```

In [48]:
```
# Display all elements from the 1st column by index
for col_num in range(len(a)):
    print(a[col_num][0])
```

```
77
96
70
```

Output the rows of the preceding two-dimensional list.

In [50]:
```
# Iterate over each row in the list or array 'a'
for row in a:
    for item in row:   # Iterate over each item in the current row
        print(item, end=' ')  # Print the current item followed by a spac
    print()  # After printing all items in the current row, move to the n
```

```
77 68 86 73
96 87 89 81
70 90 86 81
```

How the Nested Loops Execute

In [51]:
```
for i, row in enumerate(a):
    for j, item in enumerate(row):
        print(f'a[{i}][{j}]={item} ', end=' ')
    print()
```

```
a[0][0]=77  a[0][1]=68  a[0][2]=86  a[0][3]=73
a[1][0]=96  a[1][1]=87  a[1][2]=89  a[1][3]=81
a[2][0]=70  a[2][1]=90  a[2][2]=86  a[2][3]=81
```

Explanation of `enumerate()`:

enumerate(a): This generates a sequence of pairs, where each pair consists of:

- `i`: the index of the current row in the list a (starts from 0),
- `row`: the actual row (which is itself a list or sequence). This means the outer loop is iterating over both the index i and the row simultaneously.

enumerate(row): Similarly, for each row, the enumerate(row) function provides:

- `j`: the index of the current item in the row (starts from 0),
- `item`: the actual value or element in the row. So the inner loop iterates over both the index j and the item in that specific row.

Activity:

Given the following two-dimensional list of student grades:

```
grades = [[77, 68, 86, 73],  # first student's grades
          [96, 87, 89, 81],  # second student's grades
```

```
                [70, 90, 86, 81]]  # third student's grades
```

Tasks:

- Print the grades of the second student.
- Calculate and print the average grade for each student. Use a for loop.
- Print the highest grade in the entire list.

In [ ]:
```
#Print the grades of the second student.
```

In [ ]:
```
# Calculate and print the average grade for each student. Use a for loop.
```

In [ ]:
```
# Print the highest grade in the entire list.
```

FUNCTIONS

What is Function

Functions are an essential part of programming in Python. They help organize code, make it reusable, and break down tasks into smaller, more manageable parts

---

Why Functions are important

- Functions allow the same piece of code to be **reuse** multiple times .
- Functions break long programs up into smaller **manageable** components.
- Functions makes programs easier to **modify**
- Functions can be **shared** and used by other programmers.
- Functions/methods are significant **part of object-oriented programming**
- Function in Python can be assigned to a variable!

Type of Functions:

1. **Built-in** Functions
2. **User-defined** Functions
3. **Lambda** Functions
4. **Recursion** Functions

Built-in Functions

- The **print()** function prints the specified message to the screen or other standard output device.

In [52]:
```
print('Hello World')
```

```
Hello World
```

- The **input()** function takes input from the user (as a string)

In [53]:
```
input('Enter the code of your module > ')
```

```
Enter the code of your module > CST1510
```

Out[53]:
```
'CST1510'
```

- The **max()** function return the larger element

In [54]:
```
max(15, 9, 27, 14)
```

Out[54]:
```
27
```

Custam define `FUNCTIONs`

Example:

In [55]:
```
def square(number):
    """Calculate the square of number."""
    return number ** 2
```

In [56]:
```
square(7)
```

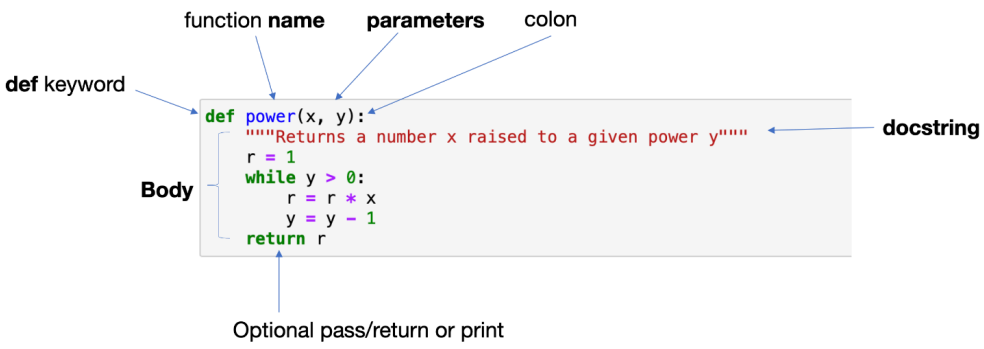
Out[56]:
```
49
```

In [57]:
```
square(2.5)
```

Out[57]:
```
6.25
```

Defining Custam `FUNCTIONs`

- A function in Python is defined using the `def` keyword, followed by the function name, a pair of parentheses `()`, and a colon `:`. The function name should follow **PEP8** naming conventions:
- Use lowercase letters.
- Separate words in multi-word names with underscores.
- Function names should be descriptive of the task they perform.



Function Structure:

In [58]:
```
def function_name(parameters):
    """Optional docstring describing the function."""
    # Function body (indented code block)
    return value  # Optional return statement
```

Example:

In [59]:
```
def greet(name):
    """Greets the person with the provided name."""
    print(f"Hello, {name}!")
```

Parameters and Arguments

- **Parameters** are variables defined in the function signature (inside the parentheses). They allow you to pass data to the function.
- **Arguments** are the actual values you provide when calling the function.

In [60]:
```
def add(a, b):
    """Returns the sum of two numbers."""
    return a + b
```

In [61]:
```
result = add(3, 5)  # 'a' becomes 3 and 'b' becomes 5
print(result)       # Output: 8
```

8

Docstrings

A docstring is a string literal that appears as the first statement in a function body. It describes what the function does and is considered a good practice, especially for larger projects. You can access a function's docstring in interactive environments like IPython or Jupyter by appending `?` to the function name.

In [62]:
```
# Example

def multiply(x, y):
    """Returns the product of two numbers x and y."""
    return x * y
```

In [63]:
```
# Access the docstring
multiply?
```

Return Statement

The `return` statement is used to send back a value from the function to its caller. If a function doesn't have a `return` statement, it implicitly returns `None`.

- With a return value:

In [65]:
```
def square(x):
    """Returns the square of a number."""
    return x * x
```

In [66]:
```
result = square(4)  # result is 16
print(result)
```

```
16
```

- Without a `return` value (returns `None` by default):

In [67]:
```
def say_hello():
    """Prints a greeting."""
    print("Hello!")
```

In [68]:
```
result = say_hello()  # result is None
print(result)
```

```
Hello!
None
```

Default Parameter Values

You can specify `default values` for parameters. If the caller does not provide a value, the default value will be used.

In [69]:
```
# Example:
```

```python
def rectangle_area(length=2, width=3):
    """Returns the area of a rectangle."""
    return length * width
```

In [70]:
```python
# Call the function with default values
print(rectangle_area())        # Output: 6 (2 * 3)
```

```
6
```

In [71]:
```python
# Override default for length
print(rectangle_area(10))      # Output: 30 (10 * 3)
```

```
30
```

In [72]:
```python
# Override both length and width
print(rectangle_area(10, 5))   # Output: 50 (10 * 5)
```

```
50
```

Keyword Arguments

When calling a function, you can use `keyword arguments` to specify values by the parameter's name. This allows passing arguments in any order.

In [73]:
```python
# Example:

def rectangle_area(length, width):
    """Returns the area of a rectangle."""
    return length * width

# Use keyword arguments
print(rectangle_area(width=5, length=10))  # Output: 50
```

```
50
```

Functions with Arbitrary Argument Lists

Python functions can accept an arbitrary number of arguments using the `*args` syntax. This collects extra arguments into a tuple.

In [74]:
```python
def average(*args):
    """Returns the average of any number of arguments."""
    return sum(args) / len(args)
```

In [75]:
```python
# Passing varying numbers of arguments
print(average(10, 20, 30))        # Output: 20.0
print(average(5, 15, 25, 35))     # Output: 20.0
```

```
20.0
20.0
```

Passing Iterable Elements as Individual Arguments

You can unpack elements of a tuple or list using the `*` operator when calling a function. This allows you to pass elements as individual arguments

In [76]:
```
grades = [90, 85, 92, 88]
print(average(*grades))  # Output: 88.75 (unpacking the list as separate
```

```
88.75
```

Functions with Multiple Parameters

Functions can have multiple parameters, and you can combine default parameters with positional arguments and keyword arguments.

In [77]:
```
def maximum(a, b, c):
    """Returns the largest of three values."""
    max_value = a
    if b > max_value:
        max_value = b
    if c > max_value:
        max_value = c
    return max_value
```

In [78]:
```
# Calling with various types of arguments
print(maximum(12, 27, 36))         # Output: 36
print(maximum('apple', 'orange', 'banana'))  # Output: 'orange'
```

```
36
orange
```

What Happens When a Function is Called

- Parameters are temporary and exist only during the function call.
- They are created when the function is called and destroyed once the function returns a value.
- All variables defined inside the function (including parameters) are local variables and cannot be accessed outside the function.

In [79]:
```
def demo_function(a, b):
    result = a + b  # 'result' is local to this function
    return result

# result is only accessible inside demo_function
```

Recursive Functions

A function can call itself, which is called recursion. Recursive functions are useful when solving problems that can be broken down into smaller, similar subproblems.

In [80]:
```
def factorial(n):
    """Returns the factorial of a number using recursion."""
    if n == 0:
        return 1
    return n * factorial(n - 1)

# Calling the recursive function
print(factorial(5))  # Output: 120 (5 * 4 * 3 * 2 * 1)
```

`pass`, `return`, and `print` Statements in the Function Body

In Python, there are various ways to handle functionality inside a function and return results to the caller. Let's look at the key statements: `pass`, `return`, and `print`.

1. `pass` Statement

- The `pass` statement is used as a placeholder when code is syntactically required but you don't want to execute anything yet. It allows the program to run without errors when an empty code block is not allowed.
- **Usage**: It's commonly used in function bodies or loops during development when you haven't implemented the logic yet.
- **Example**:
  ```
  def my_function():
      pass  # Placeholder for future code
  ```

2. `return` Statement

- The `return` statement is used to exit a function and optionally send back a value to the function's caller.
- **With an Expression**: When `return` is followed by an expression, that expression's value is returned to the caller.
  - **Example**:
    ```
    def add(a, b):
        return a + b
    result = add(2, 3)  # result is 5
    ```
- **Without an Expression**: When `return` is used without an expression, it implicitly returns `None`. This is the default behavior if a function doesn't have a `return` statement at all.
  - **None** represents the **absence of a value** and **evaluates to `False` in conditional expressions**.
  - **Example**:
    ```
    def my_function():
        return
    result = my_function()  # result is None
    ```

3. `print` Statement

- The `print` statement outputs the given data to the console but **does not return a value** to the caller.
- While `print` is useful for displaying output during development or for user interaction, it should not be confused with `return`, which sends data back to

the calling code.

- **Example**:

```
def greet(name):
    print(f"Hello, {name}!")  # Prints the greeting to the console but returns None
```

Exercise 1: Basic Function Definition

Define a function greet_user that takes one parameter, name, and prints a greeting message: "Hello, !"

In [ ]:

```

```

Exercise 2: Function with Return Value

Define a function square that takes a number as a parameter and returns its square.

In [ ]:

```

```

Exercise 3: Function with Default Parameters

Define a function rectangle_area that calculates the area of a rectangle. It should take two parameters: length and width. If no values are provided, the function should use default values of length=2 and width=3.

In [ ]:

```

```

Exercise 4: Keyword Arguments

Define a function full_name that accepts two parameters: first_name and last_name. It should return the full name formatted as "last_name, first_name". Use keyword arguments to call the function

In [ ]:

```

```

Exercise 5: Mode Calculation

Define a function mode that can accept any number of numerical arguments and return the mode of the numbers. The mode is the number that appears most frequently. If there are multiple modes, return all of them in a list.

In [ ]:

```

```

Methods: Functions Associated with Objects

- A **method** is a function that is called on an object using the syntax:

In [81]:
```
m = 'CST1510'
```

In [82]:
```
m.lower()  # call lower method on string object m
```

Out[82]:
```
'cst1510'
```

In [83]:
```
m.upper()
```

Out[83]:
```
'CST1510'
```

In [84]:
```
m
```

Out[84]:
```
'CST1510'
```

Scope Rules in Python

In Python, the `scope` of an identifier (a variable, function, or class) determines where it can be accessed within your program. Understanding scope is essential for managing the visibility and lifetime of variables in your code.

Types of Scope

1. **Local Scope:** Identifiers defined within a function or block.
2. **Global Scope:** Identifiers defined at the top level of the script or module, outside of any function or block.
3. **Non-Local Scope:** A special case used in nested functions, when accessing a variable from an outer (but not global) function.

Local Scope

A variable that is defined inside a function has **`local scope`**, meaning it can only be accessed within that function. Once the function finishes execution, the local variables are destroyed, and they cannot be accessed outside of the function.

In [85]:
```
# Example:
def local_scope_example():
    x = 10  # 'x' is local to this function
    print(f"x inside function: {x}")

local_scope_example()

# Outside the function, 'x' is not defined

# print(x)  # This would raise a NameError
```

```
x inside function: 10
```

In this example, x is local to local_scope_example and cannot be accessed outside the function.

Global Scope

A variable defined outside any function or class has **`global scope`**. Global variables are accessible throughout the entire program, including inside functions (though you cannot modify them directly without using the global keyword, which we'll discuss below).

In [87]:
```
x = 7  # Global variable

def access_global():
    print(f"x printed from access_global: {x}")
```

In [88]:
```
access_global()  # Output: x printed from access_global: 7
```

```
x printed from access_global: 7
```

Here, x is a global variable and can be accessed from within the function access_global. However, by default, you cannot modify a global variable from inside a function without explicitly declaring it as global.

Modifying Global Variables in Functions

When you try to assign a new value to a global variable inside a function without declaring it as global, Python treats it as a `local variable`. This local variable shadows the global variable of the same name, making the global variable inaccessible within that function.

In [89]:
```
x = 7  # Global variable

def try_to_modify_global():
    x = 3.5  # Local variable 'x' shadows the global variable 'x'
    print(f"x printed from try_to_modify_global: {x}")
```

In [90]:
```
try_to_modify_global()  # Output: x printed from try_to_modify_global: 3.
print(f"x outside function: {x}")  # Output: x outside function: 7
```

```
x printed from try_to_modify_global: 3.5
x outside function: 7
```

In this case, inside try_to_modify_global, the local variable x shadows the global variable. The global x remains unchanged.

To modify a global variable inside a function, you must use the `global keyword`.

In [91]:
```
# Example:

x = 7  # Global variable

def modify_global():
    global x  # Declare 'x' as global
    x = 'hello'
    print(f"x printed from modify_global: {x}")
```

In [92]:
```
modify_global()  # Output: x printed from modify_global: hello
print(f"x outside function: {x}")  # Output: x outside function: hello
```

```
x printed from modify_global: hello
x outside function: hello
```

Here, the `global` keyword allows you to modify the global variable `x` inside the function `modify_global`.

Non-Local Scope in Nested Functions

In Python, nested functions can access variables from the enclosing function's scope. If you need to modify an outer function's variable in a nested function, you must use the `nonlocal` keyword.

In [93]:
```
def outer_function():
    x = "outer"

    def inner_function():
        nonlocal x  # Allows modification of 'x' from the outer function
        x = "inner"
        print(f"x inside inner_function: {x}")

    inner_function()
    print(f"x inside outer_function: {x}")

outer_function()
```

```
x inside inner_function: inner
x inside outer_function: inner
```

In this example, `x` is defined in `outer_function`, and the `nonlocal` keyword allows the inner function to modify the variable in the outer scope.

Shadowing Functions and Variables

Python allows you to assign values to variables that may have the same name as built-in functions. This process is called shadowing. If you define a variable that has the same name as a built-in function, it will override the built-in function and make it inaccessible in your code.

In [94]:
```
# Example:
# Shadowing the built-in sum function
sum = 10 + 5
print(sum)  # Output: 15

# sum([1, 2, 3])  # This would raise a TypeError because sum is now an in
```

```
15
```

Exercise 1: Local Scope

Define a function calculate_area that calculates the area of a rectangle using two local variables, length and width. Print the area inside the function. Try accessing

length outside the function to observe what happens.

Steps:

- Inside the function, define length and width with values.
- Calculate the area and print it inside the function.
- Try printing length outside the function (it should raise an error).

In [ ]:

```

```

Exercise 2: Global Scope

Define a global variable name and create a function greet_user. Inside the function, print a message using the global variable.

Steps:

- Define a global variable name.
- Define the function greet_user that prints a greeting using the name variable.
- Call the function and observe the output.

In [ ]:

```

```

Exercise 3: Modifying Global Variables

Create a global variable counter and a function increment_counter. Inside the function, try to modify counter without using the global keyword, and observe the result. Then modify the function to properly update the global variable using global.

Steps:

- Define counter globally.
- Define the function increment_counter that tries to modify counter without global (it should not change the global variable).
- Modify the function using the global keyword to update counter.

In [ ]:

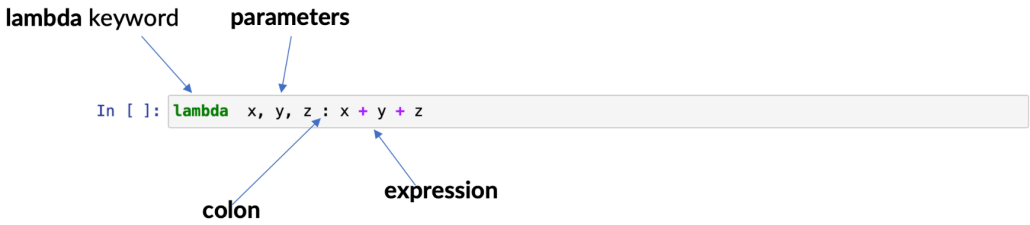
```

```

LAMBDA function

You can define small, anonymous functions using the lambda keyword. Lambda functions are useful for short, simple operations.

- A lambda expression is an _anonymous function
- Begins with the **`lambda`** keyword followed by a comma-separated parameter list, a colon (**`:`**) and an expression.
- A `lambda` *implicitly* returns its expression's value.




```
In [95]: lambda_fun1 = lambda  x, y, z : x + y + z
```

```
In [96]: lambda_fun1(1,10,100)
```

```
Out[96]: 111
```

```
In [97]: (lambda  x, y, z : x + y + z) (20,30,40)
```

```
Out[97]: 90
```

```
In [98]: (lambda  x, y, z : x ** 2 + y + (z % 2)) (2, 10, 5)
```

```
Out[98]: 15
```

```
In [99]: (lambda  x : x % 2 == 0) (10)
```

```
Out[99]: True
```

```
In [100…: (lambda  x, y, z = 3 : x + y + z) (1, 2)
```

```
Out[100…: 6
```

```
In [101…: (lambda  x, y, z = 3 : x + y + z) (1, y = 2)
```

```
Out[101…: 6
```

```
In [ ]: (lambda  *args : sum(args)) (1,2,3)
```

```
In [ ]:
```

Exercise 1: Basic Lambda Function

Create a lambda function that accepts two parameters, a and b, and returns their sum. Then, call the function with the values 5 and 10.

```
In [ ]:
```

Exercise 2: Lambda with Conditional Logic

Write a lambda function that takes a single parameter x and returns True if x is even, otherwise returns False.

```
In [ ]:
```

Exercise 3: Lambda with Multiple Parameters

Create a lambda function that accepts three parameters x, y, and z and returns the product of the three values. Call it with the values 2, 3, 4.(*product is the result of multiplying two or more numbers together*)

In [ ]:

```

```

Exercise 4: Lambda with Default Parameter

Define a lambda function that takes three parameters x, y, and z where z has a default value of 10. The function should return the sum of the three numbers. Call it with values for x and y only.

In [ ]:

```

```

Extra!

Using `filter`, `map`, `reduce` and `zip` functions

`filter()`

The function constructs an iterator from elements of an iterable for which a function returns True.

- *function:* A function that tests each element and returns True or False.
- *iterable:* The sequence (e.g., list, tuple) to be filtered.

Example:

In [102…

```
# Define a list of numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Define a function to check if a number is odd
is_odd = lambda x: x % 2 != 0

# Use filter to get only odd numbers
odd_numbers = list(filter(is_odd, numbers))

print(odd_numbers)  # Output: [1, 3, 5, 7, 9]
```

```
[1, 3, 5, 7, 9]
```

`map()`

The map() function applies a given function to each item of an iterable and returns a list of the results.

- *function:* The function to apply.
- *iterable:* The sequence to process.

In [103…

```
# Define a list of numbers
numbers = [1, 2, 3, 4, 5]

# Define a function to double a number
double = lambda x: x * 2

# Use map to apply the function to each number
doubled_numbers = list(map(double, numbers))

print(doubled_numbers)  # Output: [2, 4, 6, 8, 10]
```

```
[2, 4, 6, 8, 10]
```

`reduce()`

The reduce() function applies a function cumulatively to the items of an iterable, reducing the iterable to a single value. This function is available in the functools module.

- *function:* The function to apply (must accept two arguments).
- *iterable:* The sequence to process.

In [104…

```
from functools import reduce

# Define a list of numbers
numbers = [1, 2, 3, 4, 5]

# Define a function to multiply two numbers
multiply = lambda x, y: x * y

# Use reduce to get the product of all numbers
product = reduce(multiply, numbers)

print(product)  # Output: 120
```

```
120
```

`zip()`

The `zip()` function takes multiple iterables (e.g., lists, tuples) and combines their elements into tuples, with each tuple containing one element from each iterable. It stops when the shortest input iterable is exhausted.

In [105…

```
# Define two lists: names and ages
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]

# Use zip to combine them into pairs
combined = list(zip(names, ages))

print(combined)  # Output: [('Alice', 25), ('Bob', 30), ('Charlie', 35)]
```

```
[('Alice', 25), ('Bob', 30), ('Charlie', 35)]
```

Extra Activity: Practice with filter(), map(), reduce(), and zip() Functions

Task 1: Use `filter()` to Get Even Numbers

Define a list of numbers from 1 to 20. Use the filter() function to create a new list that contains only the even numbers from the original list.

In [ ]:

```

```

Task 2: Use `map()` to Square Each Number

Using the same list of numbers from 1 to 10, use the map() function to create a new list that contains the square of each number.

In [ ]:

```

```

Task 3: Use `reduce()` to Find the Sum of Numbers

Using the reduce() function, calculate the sum of all numbers in a list from 1 to 10. Use reduce from the functools module.

In [ ]:

```

```

Task 4: Use zip() to Combine Two Lists

You are given two lists:

```
cities = ["New York", "Los Angeles", "Chicago"]
```
and

```
populations = [8175133, 3792621, 2695598]
```

- Use the zip() function to combine them into pairs where each city is paired with its population.

In [ ]:

```

```